INC-001 — Detailed Findings Report

Incident	INC-001 — SSH brute force leading to account compromise and persistence
Classification	Lab exercise, sanitized for publication
Activity date	2026-10-30
Report date	2026-10-31
Author	SOC L1 portfolio lab
Status	Closed — escalated
Severity	High
Related	`writeups/INC-001-ssh-brute-force.md` (narrative), `tickets/INC-001-ssh-brute-force.md` (ticket), `docs/evidence/`

---

1. Executive summary

At 15:21:47 UTC, a legitimate session by the service account `svc-backup` established a baseline on the Debian endpoint `soc-victim`. Three seconds later, a password-guessing tool began submitting passwords for the same account. Twenty attempts later, at 15:22:21 UTC, one succeeded. One second after that, the account's `~/.ssh/authorized_keys` file was created, giving the attacker a way back in that would survive a password reset. Discovery commands followed immediately.

The full sequence, from launching the tool to a working login, took **34 seconds**. From the first failed password to persistence took **32 seconds**.

Wazuh detected the activity at three stages:

Time (UTC)	Rule	Meaning
15:22:01	**5763** (level 10)	sshd brute force
15:22:21	**40112** (level 12)	Multiple auth failures followed by a success
15:22:22	**100120** (level 12)	FIM: `authorized_keys` added or modified

The detection was not the difficult part. The environment produced **400 alerts in 150 seconds**, 292 of them audit command events. Stock rule 40112 also fired on **six alerts from four routine analyst sessions**, because every session in the lab shares the virtual gateway address `10.0.2.2`. Separating the real compromise from the noise is the core of this report.

The root cause is a configuration failure, not a sophisticated attack: a service account had password authentication enabled, a password that appeared in a 20-entry list, no lockout, and permission to open an interactive shell.

---

2. Scope and methodology

2.1 Scope

In scope	Out of scope
One Debian 12 VM (`soc-victim` , 10.0.2.15)	Production systems
Wazuh manager, indexer, dashboard on the host	Any external network
Endpoint telemetry: journald, auditd, FIM	Real malware or C2 infrastructure
Attacker: `hydra` from the QEMU host	Employer, client, or government data

2.2 Method

1. Build a reproducible endpoint with the Wazuh agent and the log sources the role description names.
2. Generate legitimate background activity (analyst logins, `sudo` , web requests) so the attack is not the only thing in the queue.
3. Execute a password-guessing attack against a service account, then simulate post-exploitation with discovery and SSH-key persistence.
4. Let the SIEM collect without intervention, then reconstruct the incident from the alerts, journal, audit records, and FIM events.
5. Write the ticket and this report from the evidence, not from the attack script.

2.3 Limitations

- **Source attribution.** QEMU user networking presents every connection as the gateway `10.0.2.2` . Legitimate and malicious sessions share one address. A real estate would not have this exact problem, but shared NAT and jump hosts produce a milder version of it.
- **Scale.** One endpoint. No domain controller, no cloud telemetry, no email gateway.
- **Synthetic data.** The account, password, and key were created for the exercise.
- **Timing.** FIM events are batched; the alert timestamp (15:22:22.000) is the batch time, not the exact write time (15:22:21).

---

3. Environment

Component	Detail
SIEM	Wazuh 4.14.7, single-node Docker (`wazuh.manager`, `wazuh.indexer`, `wazuh.dashboard`)
Endpoint	Debian 12 genericcloud, 1.5 GB RAM, 2 vCPU
Agent	Wazuh agent 4.14.7, ID 001, name `soc-victim`
Auth telemetry	sshd → journald, forwarded to the manager
Process telemetry	auditd, key `audit-wazuh-c` (execve)
File telemetry	FIM realtime on `/etc`, `/home`, `/var/www/html`
File watches	auditd `audit-wazuh-w` on identity files, `cron.d`, webroot
Network sensor	Suricata 7.0.10, 8,234 ET Open rules on `ens3` (not relevant to SSH)
Attacker	`hydra` on the host, 20-password list, 2 threads
Clock	All systems UTC

Network path: host `10.0.2.2` → forwarded port `2222` → guest `10.0.2.15:22`.

4. Attack narrative

4.1 Preparation

A service account was created to represent a backup process that should never log in interactively:

```
sudo useradd -m -s /bin/bash svc-backup
echo 'svc-backup:Summer2026!' | sudo chpasswd
printf 'PasswordAuthentication yes\n' | sudo tee /etc/ssh/sshd_config.d/00-lab-password.conf
sudo systemctl restart ssh
```

This is the misconfiguration being tested. Everything that follows is a consequence.

4.2 Execution

Phase	Action
Baseline	Legitimate `svc-backup` login at 15:21:47
Credential access	`hydra -l svc-backup -P passwords.txt -s 2222 -t 2 127.0.0.1 ssh`
Initial access	Password `Summer2026!` accepted at 15:22:21
Discovery	`id`, `whoami`, `uname -a`, `head -1`, `sudo -n -l`
Persistence	Append an SSH public key to `~/.ssh/authorized_keys`
Egress check	`curl http://10.0.2.2:8080/` → HTTP 200

4.3 Kill chain (MITRE ATT&CK)

Tactic	Technique	ID	Evidence
Reconnaissance	Active scanning (light, by the tool)	T1595	Connection attempts
Credential Access	Brute force: password guessing	T1110.001	20 × 5760, then 5763
Initial Access	Valid accounts	T1078	Password login accepted
Execution	Unix shell	T1059.004	`bash -c` script
Discovery	System information discovery	T1082	`uname -a`
Discovery	System owner/user discovery	T1033	`id`, `whoami`
Persistence	Account manipulation: SSH authorized keys	T1098.004	FIM 100120
Command and Control	Application layer protocol (HTTP)	T1071.001	`curl` to 10.0.2.2:8080

No privilege escalation succeeded (`sudo` required a password), and no lateral movement was observed.

---

5. Timeline

Time (UTC)	Source	Event
15:21:47	auth.log	Legitimate `svc-backup` login from 10.0.2.2
15:21:49	Wazuh 5715	Alert for that login (ingest delay)
15:21:50	auth.log	First failed password
15:21:51	Wazuh 5760	First failure alert
15:22:01	Wazuh **5763**	Brute force recognised (8 failures / 120 s threshold)
15:22:03	Wazuh 40112	**False positive** — analyst login
15:22:05	Wazuh 40112	**False positive** — analyst login
15:22:19	auth.log	20th failure
15:22:21	auth.log	`Accepted password for svc-backup`
15:22:21	Wazuh **40112**	Success after failures, `svc-backup`
15:22:22	Wazuh **100120**	`authorized_keys` added (FIM, realtime)
15:22:23	auditd	`bash -c` script, `id`, `whoami`, `uname`, `head`, `sudo -n -l`, `mkdir`, `curl`
15:22:23	auth.log	Two in-flight guesses still arriving
15:22:35	Wazuh 40112	False positive — analyst login
15:23:05	Wazuh 40112	False positive — analyst login

The 32-second window from first failure to persistence is the number that matters. An account lockout, a key-only policy, or a five-minute triage SLA would each have changed the outcome.

---

6. Evidence

6.1 Alert volume in the incident window

Window: 15:21:00–15:23:30 UTC, agent `soc-victim`.

Rule	Description	Count
80792	Audit: command executed	292
5501	PAM: login session opened	27
5502	PAM: login session closed	23
5760	sshd: authentication failed	20
40112	Multiple auth failures followed by a success	8
5715	sshd: authentication success	6
5763	sshd: brute force	1
100120	**FIM: authorized_keys added or modified**	**1**
	Total	**400**

The signal-to-noise ratio is roughly 1 in 400 by alert count. The practical filter is account and time, not severity alone.

6.2 Authentication log

```
Oct 30 15:21:47 soc-victim sshd[10751]: Accepted password for svc-backup from 10.0.2.2 port 49820
ssh2
Oct 30 15:21:50 soc-victim sshd[10779]: Failed password for svc-backup from 10.0.2.2 port 49856 ssh2
Oct 30 15:21:50 soc-victim sshd[10778]: Failed password for svc-backup from 10.0.2.2 port 49842 ssh2
...
Oct 30 15:22:19 soc-victim sshd[10821]: Accepted password for svc-backup from 10.0.2.2 port 55468
ssh2
Oct 30 15:22:21 soc-victim sshd[10819]: Failed password for svc-backup from 10.0.2.2 port 55464 ssh2
Oct 30 15:22:21 soc-victim sshd[10847]: Accepted password for svc-backup from 10.0.2.2 port 52318
ssh2
```

The `Accepted` at 15:22:19 in the journal is the login shell for the earlier probe; the successful interactive session is 15:22:21. Two workers were still guessing after the first success, which is normal for parallelised tools.

6.3 File integrity evidence


```
{
  "timestamp": "2026-10-30T15:22:22.000+0000",
  "rule": {
    "id": "100120",
    "level": 12,
    "description": "FIM: SSH authorized_keys added or modified (possible persistence)",
    "mitre": { "id": ["T1098.004", "T1078"] }
  },
  "syscheck": {
    "path": "/home/svc-backup/.ssh/authorized_keys",
    "mode": "realtime",
    "event": "added",
```

```
    "uid_after": "1002",
    "sha256_after": "61954205be034570c75bdd6db67b3fbd3410ac1d4bebb08b65d117667965c5e8"
  }
}
```

6.4 Process evidence (decoded)

```
15:22:21Z  bash -c <script>
15:22:21Z  id
15:22:21Z  whoami
15:22:21Z  uname -a
15:22:21Z  head -1
15:22:21Z  sudo -n -l
15:22:21Z  mkdir -p /home/svc-backup/.ssh
15:22:21Z  curl -s -o /dev/null -w outbound_http=%{http_code} http://10.0.2.2:8080/
```

The `bash -c` argument decoded to a script that performed discovery, the `sudo` check, the key append, and the outbound request. `sudo -n -l` returned "a password is required", confirming no privilege escalation.

Full evidence: `docs/evidence/INC-001-{alerts.json,auth.log,audit.log,postex-commands.txt}`.

---

7. Detection analysis

7.1 What fired, and how fast

Rule	Trigger	Latency	Value
5760	One failed password	<1 s	Volume; not actionable alone
5763	8 failures / 120 s	11 s after first failure	Good early warning
40112	Any auth failure + success, same IP	Immediate on success	Right idea, too broad
100120	FIM add/modify of `authorized_keys`	<1 s after write	High-fidelity persistence signal

7.2 Why 40112 produced false positives

Stock 40112 correlates:

```
<rule id="40112" level="12" timeframe="240">
  <if_group>authentication_success</if_group>
  <if_matched_group>authentication_failures</if_matched_group>
  <same_source_ip />
  <description>Multiple authentication failures followed by a success.</description>
</rule>
```

It does not require the success to be for the same **account**, and it does not distinguish key-based from password-based authentication. With every session arriving from `10.0.2.2`, any analyst login after the spray

matched. Six of the eight 40112 alerts in the window came from four routine analyst sessions.

A narrower rule would require:

- the success to use password authentication,
- the account to match the one being guessed,
- a minimum failure count,
- a source address that is not a known jump host.

The first two are the hard part in Wazuh. Failures expose `srcuser`; successes expose `dstuser`, and `<same_user />` does not bridge them. This is documented in `03-detection-mitigation-triage.md` and `rules/tuning-notes.md`.

7.3 What did not fire

Gap	Why it matters
No alert for interactive login by a service account	The account should never open a shell
No lockout event	20 attempts were allowed
No Suricata alert	SSH is encrypted; network IDS cannot see the guesses
No rule for a new SSH key	Closed by custom rule 100120
No correlation between key write and the preceding login	The analyst had to join them manually

---

8. Root cause

8.1 Direct cause

A service account was configured with:

- **password authentication enabled**, contrary to the principle that non-human accounts use keys only;
- **a weak, guessable password** (`Summer2026!`) present in a 20-entry list;
- **an interactive shell** (`/bin/bash`) and a home directory it did not need;
- **no account lockout**, so 20 attempts were free.

8.2 Contributing factors

Factor	Effect
`MaxAuthTries` default (6 per connection)	Allowed 20 attempts across connections
No `pam_faillock`	No lockout or delay
No MFA	Password was the only factor
Monitoring rule 40112 is account-blind	Noise obscured the real success
Full execve auditing	292 alerts in 150 s, hiding the useful ones
Shared gateway address	Prevented source-based separation

8.3 Five whys

1. **Why was the host compromised?** A password was guessed for a valid account.
2. **Why was guessing possible?** Password authentication was enabled, with no lockout.
3. **Why was the account exposed?** A service account had an interactive shell and a password, and no policy prevented it.
4. **Why did nobody stop it?** The detection that fired (40112) also fired on benign logins, and the environment was noisy.
5. **Why was the noise tolerated?** Auditing was enabled without tuning, and the lab accepted the default rule behaviour. The fix is a policy and tuning problem, not a tool problem.

---

9. Impact

Dimension	Assessment
Confidentiality	Account-level access; discovery commands. No evidence of data exfiltration.
Integrity	`authorized_keys` modified; an attacker-controlled credential was installed.
Availability	None observed.
Blast radius	One host in the lab. In production, the account's reachable systems determine this.
Persistence	High: the key survives a password reset and account re-enablement.

The most important impact is not the initial access; it is the **persistence**. A response that only resets the password leaves the attacker a working key.

---

10. Severity

High.

- **Not Critical:** the account was unprivileged, `sudo` failed, no new accounts, no lateral movement, one host.

- **Not Medium:** a valid credential and a persistent key were established. The attacker can return after a password reset if the key is missed.

I deliberately did not assign a CVSS base score. A base vector treats all unauthenticated network attacks alike and ignores the account's privilege level, the internal exposure, and the failed escalation. The operational rating is the one that should drive the response.

---

11. Response actions

Performed as part of the exercise:

1. Isolated the host (the lab network is host-only; snapshot preserved).
2. Disabled `svc-backup` and expired the password.
3. Removed the unauthorised key **after** recording its SHA-256.
4. Preserved alerts, journal, audit records, and the decoded command list.
5. Escalated with a timeline, the hash, and the failed-escalation note.
6. Verified no new accounts, no sudoers change, no cron persistence (the `crontab` binary was absent, and the attempt failed).

---

12. Recommendations

Immediate (before the next shift)

#	Action	Why
1	Disable password authentication for all service accounts	Removes the entry vector
2	Give non-interactive accounts `/usr/sbin/nologin`	Removes the shell
3	Remove the key and rotate the account's credentials	Removes persistence
4	Enable `pam_faillock`	Stops 20 free guesses
5	Baseline `authorized_keys` fingerprints per account	Turns FIM into a real control

Short term (this sprint)

#	Action
6	Tune 40112 to sshd authentication groups and raise the failure threshold
7	Add a rule for successful login by a service account
8	Reduce auditd noise: audit command categories, not every `execve`
9	Add a change window or allow-list for legitimate key rotation
10	Document the SSH exposure and restrict it to a management network or jump host

Long term

#	Action
11	MFA for all interactive SSH
12	Centralised key management with expiry and fingerprint inventory
13	Detection-as-code tests for 5763/40112/100120
14	Measure MTTD/MTTA against a five-minute objective
15	Tabletop the "key survives password reset" scenario

---

13. Lessons learned

- **Persistence changes the response.** The password is not the incident; the key is.
- **A noisy rule is worse than no rule** when it trains analysts to ignore it.
- **Source IP is not identity.** Shared NAT, jump hosts, and proxies break IP-only correlation.
- **Detection latency was fine.** The failure was policy and triage, not tooling.
- **Audit everything is not monitoring.** 292 command alerts in 150 seconds is collection, not detection.

Appendix A — Custom rule

```
<rule id="100120" level="12">
  <if_sid>550,554</if_sid>
  <match>authorized_keys</match>
  <description>FIM: SSH authorized_keys added or modified (possible persistence)</description>
  <mitre>
    <id>T1098.004</id>
    <id>T1078</id>
  </mitre>
  <group>syscheck,persistence,</group>
</rule>
```

Test:

```
sudo -u svc-backup bash -c 'echo "ssh-ed25519 AAAA... lab" >> ~/.ssh/authorized_keys'
# expect rule 100120 within seconds, with syscheck.path and sha256_after
```

Appendix B — Evidence index

File	Contents
`docs/evidence/INC-001-alerts.json`	400 alerts from the incident window
`docs/evidence/INC-001-auth.log`	sshd journal, 15:21:40–15:22:30
`docs/evidence/INC-001-audit.log`	audit execve records for the compromised session
`docs/evidence/INC-001-postex-commands.txt`	Decoded command line with timestamps

Appendix C — Glossary

Term	Meaning
FIM	File integrity monitoring
MTTD	Mean time to detect
MTTA	Mean time to acknowledge
TP / FP	True positive / false positive
`srcuser` / `dstuser`	The account attempting authentication / the authenticated account
FIM realtime	Inotify-based monitoring, alerting on write rather than on a scheduled scan